

MINI PROJECT

NAME : SANJEEV N

DATE : 28-08-2025

Curl

```
curl -X 'POST' \
  'https://localhost:7114/Login?_userName=Dharanesh&_password=Dharanesh%40123' \
  -H 'accept: */*' \
  -d ''
```

Request URL

https://localhost:7114/Login?_userName=Dharanesh&_password=Dharanesh%40123

Server response

Code	Details
200	Response body <pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3bWV1Ijo1RGhhcmFuZDNoIiwiaHR0cDovL3NjaGVtYXMuYm91bm9zbnZ0LmNvbS93cy8yMDA4LzA2L2lkZW50aXR5L2NsYWltcy9yb2x1Ijo1OHRtam41lC3leHAiOiJlE3NTU4NTgwIjQsImZcyI6Imh0dHBzOi8vd3d3LnBheW9kYS5jb20vIiw1YXVkiJo1Q3VzdG9tZXJzX0NTaGFycCJ9.HMSKg0sNs2B8TINixT8Cr1QXhf8AfwxJA_Kq=M8s" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 22 Aug 2025 09:50:27 GMT server: Kestrel</pre>

Responses

Code	Description	Links
200	OK	No links

Curl

```
curl -X 'POST' \
  'https://localhost:7114/getOverBalanceOfUser?cus_name=Dharanesh' \
  -H 'accept: */*' \
  -d ''
```

Request URL

https://localhost:7114/getOverBalanceOfUser?cus_name=Dharanesh

Server response

Code	Details
401 <i>Undocumented</i>	Error: response status is 401 Response headers <pre>content-length: 0 date: Fri, 22 Aug 2025 09:51:04 GMT server: Kestrel www-authenticate: Bearer</pre>

Responses

Code	Description	Links
200	OK	No links

[illegible]

```
https://localhost:7114/GetOverBalanceOfUser?cus_name=Dharanesh
```

Code	Details
------	---------

Response body

```
12000
```

 [Download](#)

```
content-type: application/json; charset=utf-8
date: Fri, 22 Aug 2025 09:52:44 GMT
server: Kestrel
```

Code	Description	Links
------	-------------	-------

```
curl -X 'GET' \  
  'https://localhost:7114/GetCustomerDetails' \  
  -H 'accept: */*' \  
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpvcC9yLWVudWIjOiJ0RGhhcmFuZDkiLCJpdwIiwiaHR8cDovLnNjaGVtYXMuYmwlIjpm9zb2Z0LmNvbnB9cy8yMDA4LzAzL2lkZW50eSRLNSl2eWltcy9yb2xiIjoq
```

```
https://localhost:7114/GetCustomerDetails
```

Code	Details
------	---------

```
Response body
{
  "customerId": 1,
  "customerName": "dharanesh",
  "customerAge": 20,
  "createdDate": "0001-01-01",
  "accountNumber": 1002,
  "balance": 10000
},
{
  "customerId": 2,
  "customerName": "Karthi",
  "customerAge": 21,
  "createdDate": "0001-01-01",
  "accountNumber": 1003,
  "balance": 20000
}
```

```
curl -X 'POST' \  
  'https://localhost:7114/AddCustomerDetails?cus_name=arun&cus_age=34' \  
  -H 'accept: */*' \  
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyYXV1Ijo1R6hhcmFueXNoIiwiaHR0cDovL3NjaGVtYXMuYm91IjpmZ2Z0LmNvbS93cy8yMDA4LzA2L2lkZW50aXR5L2NsYWltcy9yY2ZlIjo1Q0' \  
  -d ''
```

```
https://localhost:7114/AddCustomerDetails?cus_name=arun&cus_age=34
```

Code	Details
------	---------

Response body

```
Added successfully
```

 [Download](#)

```
content-type: text/plain; charset=utf-8
date: Fri, 22 Aug 2025 09:54:00 GMT
server: Kestrel
```

```
curl -X 'POST' \
'https://localhost:7114/CreateCustomerAccount?acc_num=122&bal=20000&date=%222025-08-22%22&cus_id=4' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyYWR1Ijo1RGlhbmFuZmVkaHR0cDovLjNjaGVtYXUubWU1Ijcm9zb2Z0LmVhbnV5S3cybWMA4LzA2L2lkZW50aXR5L2NsYWltcy9yb2xi1joiQ' \
-d ''
```

https://localhost:7114/CreateCustomerAccount?acc_num=122&bal=20000&date=%222025-08-22%22&cus_id=4

Code	Details
------	---------

Added successfully

```
content-type: text/plain; charset=utf-8
date: Fri, 22 Aug 2025 09:55:03 GMT
server: Kestrel
```

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "sqlite:///./users.db"

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})

SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
```

```
Base = declarative_base()
```

```
from sqlalchemy import Column, Integer, String
from database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True, nullable=False)
    password = Column(String, nullable=False)
    role = Column(String, nullable=False)
```

```
from pydantic import BaseModel
```

```
class UserLogin(BaseModel):
    username: str
    password: str

class UserResponse(BaseModel):
    username: str
    role: str
```

```
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
import database, models, schemas, crud

models.Base.metadata.create_all(bind=database.engine)

app = FastAPI()

def get_db():
    db = database.SessionLocal()
    try:
        yield db
```

```

        finally:
            db.close()

@app.post("/login", response_model=schemas.UserResponse)
def login(user: schemas.UserLogin, db: Session = Depends(get_db)):
    db_user = crud.get_user_by_username(db, user.username)
    if not db_user or db_user.password != user.password:
        raise HTTPException(status_code=401, detail="Invalid username or password")
    return {"username": db_user.username, "role": db_user.role}

```

C# CODE

```

using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AspNetBankApp.Models
{
    public class BankAccount
    {
        [Key]
        public int AccountId { get; set; }

        [Required]
        [Column(TypeName = "TEXT")]
        public string AccountNumber { get; set; } = string.Empty;

        [Required]
        public decimal Balance { get; set; }

        public DateTime CreatedDate { get; set; } = DateTime.Now;

        public int CustomerId { get; set; }

        [ForeignKey("CustomerId")]
        public Customer? Customer { get; set; }
    }
}

```

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AspNetBankApp.Models
{
    public class Customer
    {
        [Key]
        public int CustomerId { get; set; }

        [Required]
        [Column(TypeName = "TEXT")]
        public string Name { get; set; } = string.Empty;

        public int Age { get; set; }
        public ICollection<BankAccount> BankAccounts { get; set; } = new
List<BankAccount>();
    }
}

```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using AspNetBankApp.Data;
using AspNetBankApp.Models;

namespace AspNetBankApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AccountController : ControllerBase
    {
        private readonly AppDbContext _context;

        public AccountController(AppDbContext context)
        {
            _context = context;
        }

        [Authorize(Roles = "admin")]
        [HttpPost("create")]
        public IActionResult CreateAccount(BankAccount account)
        {

```

```

        _context.BankAccounts.Add(account);
        _context.SaveChanges();
        return Ok(account);
    }

    [Authorize]
    [HttpGet("balance/{username}")]
    public IActionResult GetBalance(string username)
    {
        var accounts = _context.BankAccounts
            .Where(a => a.Customer.Name == username)
            .ToList();

        if (!accounts.Any())
            return NotFound("No accounts found for this user");

        var total = accounts.Sum(a => a.Balance);
        return Ok(new { Username = username, TotalBalance = total });
    }
}

```

```

using Microsoft.AspNetCore.Mvc;
using System.Net.Http;
using System.Text;
using System.Text.Json;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;

namespace AspNetBankApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        private readonly IConfiguration _config;
        private readonly HttpClient _httpClient;
        private readonly JsonSerializerOptions _jsonOptions;

        public AuthController(IConfiguration config, IHttpClientFactory httpClientFactory)
        {

```

```

    _config = config;
    _httpClient = httpClientFactory.CreateClient();

    _jsonOptions = new JsonSerializerOptions
    {
        PropertyNameCaseInsensitive = true,
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase
    };
}

[HttpPost("login")]
public async Task<ActionResult> Login([FromBody] LoginRequest request)
{
    var json = JsonSerializer.Serialize(request, _jsonOptions);
    var content = new StringContent(json, Encoding.UTF8, "application/json");

    var response = await _httpClient.PostAsync("http://127.0.0.1:8000/login", content);

    if (!response.IsSuccessStatusCode)
        return Unauthorized("Invalid credentials from FastAPI");

    var result = await response.Content.ReadAsStringAsync();
    var userData = JsonSerializer.Deserialize<LoginResponse>(result, _jsonOptions);

    if (userData == null)
        return Unauthorized("Invalid response from FastAPI");

    var claims = new[]
    {
        new Claim(ClaimTypes.Name, userData.Username),
        new Claim(ClaimTypes.Role, userData.Role)
    };

    var key = new
    SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"],
        audience: _config["Jwt:Audience"],
        claims: claims,
        expires: DateTime.Now.AddHours(1),
        signingCredentials: creds
    );
}

```



```

        return Ok(new { token = new JwtSecurityTokenHandler().WriteToken(token) });
    }
}

public record LoginRequest(string Username, string Password);

public record LoginResponse(string Username, string Role);
}

using Microsoft.EntityFrameworkCore;
using AspNetBankApp.Models;

namespace AspNetBankApp.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) {

            public DbSet<Customer> Customers { get; set; }
            public DbSet<BankAccount> BankAccounts { get; set; }
        }
    }
}

using AspNetBankApp.Data;
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>

```

```

{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
    };
};

builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "AspNetBankApp API", Version = "v1"
});

    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Enter 'Bearer' [space] and then your valid JWT token.\n\nExample:
\"Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...\"";
    });

    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
};

```

```
});
```

```
builder.Services.AddHttpClient();  
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
if (app.Environment.IsDevelopment())  
{  
    app.UseSwagger();  
    app.UseSwaggerUI();  
}
```

```
app.UseHttpsRedirection();
```

```
app.UseAuthentication();  
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```